
Which backend should draw which scientific visual?

Week 1 · one semantic figure spec, rendered by TikZ, Draw.io and native Beamer layout.

Diagram backends

Weekly research update · synthetic demo · version 0.1

What stays constant across renderers

ESTABLISHED

- **Method model** One semantic model of the method
- **Figure spec** Nodes
- **Routing** The renderer is chosen explicitly or by complexity heuristics.

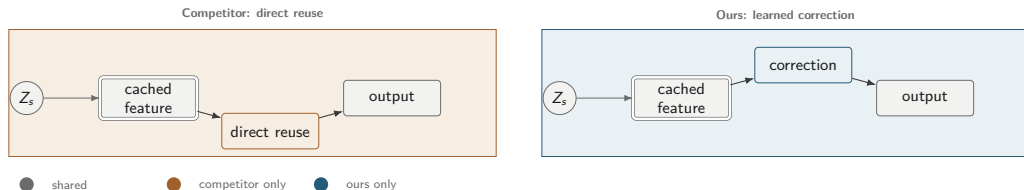
THIS WEEK

Four renderers coexist in one deck. Conceptual diagrams are native TikZ, the large architecture stays in Draw.io, results stay a native table, and the deck is still one Beamer PDF.

Previous: separate figure pipelines

Direct reuse vs learned correction

The difference should be visible in under five seconds.



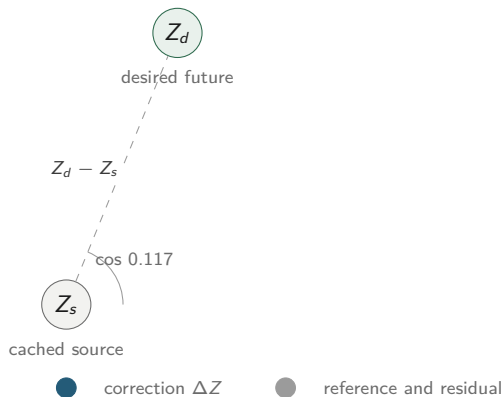
Same inputs, same cache, same output; only the operation between cache and output differs. TikZ keeps both panels in one coordinate system.

Correction direction vs desired direction



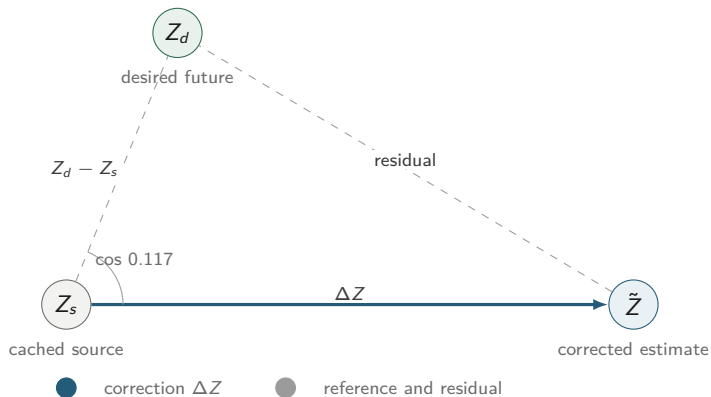
The correction is nearly orthogonal to the desired direction; the angle is annotated from the same measurement as the diagnostic slide.

Correction direction vs desired direction



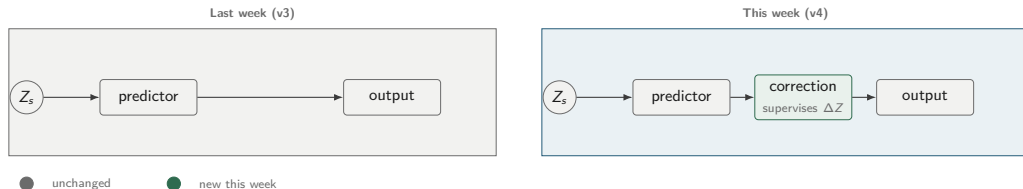
The correction is nearly orthogonal to the desired direction; the angle is annotated from the same measurement as the diagnostic slide.

Correction direction vs desired direction



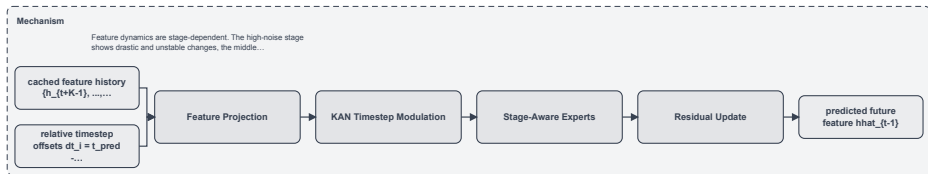
The correction is nearly orthogonal to the desired direction; the angle is annotated from the same measurement as the diagnostic slide.

Method delta: v3 to v4



Unchanged components keep their layer across both panels and stay muted; the inserted correction is the only highlighted object.

Large architecture stays editable in Draw.io



Draw.io remains the backend for large editable architecture figures; Beamer includes the exported PDF directly.

Renderer cost and editability

Method	Compile (s)	Native typography	Editable source
TikZ conceptual (this week)	1.20 (once)	1 (yes)	0.50 (tex)
Draw.io architecture (last week)	0.40 (pdf)	0.50 (via pdf)	1 (gui)
Python plot	0.30 (pre)	0.50 (no)	0.50 (script)

lower is better; 1 = native; 1 = full TikZ costs a compile step and pays back in native typography and direct Beamer integration; Draw.io stays best for large editable figures.

Limitations and open questions

CURRENT LIMITATIONS

- TikZ routing is heuristic; the decision is recorded but not yet learned from feedback.
- Only conceptual and diagnostic archetypes are implemented in v0.1.
- PGFPlots is optional and not used by the default plot pipeline.
- Overlays inside a scaled figure are not supported.

OPEN QUESTIONS

- Should routing learn from critic outcomes across weeks?
- Can the same figure spec drive a paper-ready export automatically?
- When does a TikZ diagram become too large to stay readable in a slide?